A BM TECHNICAL MANUAL

Pi Zero W and bm3d

Raspberry Pi Zero W 1.1 with bm: ARM11 and GPU performance, and the bm3d 2D and 3D drivers.

Contents

The board 4

The processor 4

Memory 4

The clocks 5

The VideoCore IV GPU 5

ARM11 performance, as measured 6

Basic costs 6

How many objects fit in a frame 6

Cost per pixel of the ARM renderer 7

QEMU is not the Pi 7

Gaps in the ARM data 7

The GPU and the 2D 8

How bm draws in 2D 8

Resolutions and the scaler 8

What gfx16 offers 8

DMA 9

2D drawn by the GPU 9

The GPU and the 3D 10

The ARM renderer (r3d) 10

The V3D path (gpu3d) 10

When each is used 10

Options 11

What the GPU gives, as measured 12

The bm3d drivers 14

One name, two drivers 14

The versions that matter 14

The 2D side 15

The 3D side 15

How a game chooses 15

Measure on your own board 17

The 3D Bench 17

A part of the bench, from the monitor 17

The cost of a pixel 18

The GPU test and the stress test 18

Compare the drivers on the same board 18

No report for a stopped test 18

If the current numbers are missing 19

Limits and tips 20

Limits 20

Tips 20

The RGB30, where the repo compares 21

Open points 22

README

What this is. What the Raspberry Pi Zero W 1.1 can do under bm, and what bm's drivers give your game: the ARM1176 processor, the VideoCore IV GPU, the 2D drawing and the 3D drivers called bm3d.

How to read it. Every number comes from a file of the repository, named next to it in a `code` span such as `docs/PERFORMANCE.md`. Chapter 6 gives the steps to measure the same things on your own board.

What is missing. Every gap is marked with a highlighted TO FILL tag and listed in the last chapter. Nothing is invented.

The board

The Pi Zero W 1.1 is the first target of bm. bm runs on it with no operating system: the game, the graphics, the audio and the input share one core.

The processor

Item	Value	Source
SoC	BCM2835	`docs/HARDWARE.md` (section 7: same SoC as the Pi 1)
CPU	ARM1176JZF-S (ARMv6), 1 core	`docs/HARDWARE.md`
Clock	700 MHz at boot, 1000 MHz requested from the firmware and measured	`docs/HARDWARE.md`
FPU	VFPv2, double in hardware; bm turns on RunFast	`docs/HARDWARE.md`
Cache	L1 16 KiB instructions + 16 KiB data; L2 128 KiB in the VideoCore	`docs/HARDWARE.md`
Integer division	not in hardware (a libgcc routine): avoid it in hot loops	`docs/HARDWARE.md`
MMU, caches, branch prediction	on, with the clock at 1000 MHz	`docs/HARDWARE.md`

Memory

Item	Value	Source
Total RAM	512 MiB LPDDR2, shared by CPU and GPU	`docs/HARDWARE.md`
RAM for the ARM	448 MiB (with `gpu_mem=64`)	`docs/HARDWARE.md`
RAM for the GPU	64 MiB; the framebuffer 640x360x4x2 takes 1.8 MiB	`docs/HARDWARE.md`
Memory for Lua	limit 64 MiB	`docs/HARDWARE.md`
Copy bandwidth	memcpy about 100 MB/s, fill about 430 MB/s	`docs/HARDWARE.md`, `docs/PERFORMANCE.md`

Item	Value	Source
Reads from SDRAM (ARM)	about 4 times slower than writes	`docs/PERFORMANCE.md`

Memory is not the limit. The time per frame and the bandwidth to RAM are (source: `docs/HARDWARE.md`).

The clocks

The repo records these clocks from a real Pi Zero W (source: `docs/bench/20261005-082943_bench3d_pi-zero-w_v0.2.0.txt`, line `machine`):

Clock	MHz
ARM	1000
core (L2 cache and memory bus)	250
V3D (the 3D unit)	250
SDRAM	400

- The core clock can go to 400 MHz at most (source: `docs/STRESS.md`).
- `enable_uart=1` in `boot/config.txt` keeps the core at 250 MHz; the V3D has also been seen at 250 MHz (source: `docs/BENCH3D.md`).
- `v3d_clock=max` in `bm/config.txt` asks the firmware for its maximum at boot. It is a trial, off by default (source: `docs/BENCH3D.md`).
- [TO FILL: the effect of `force_turbo=1` or of the 400 MHz core clock on the 3D Bench; `docs/STRESS.md` says "to be tried"]

The VideoCore IV GPU

Item	Value	Source
Core clock	250 MHz	`docs/HARDWARE.md`
3D unit (V3D)	12 QPUs in 3 slices, filtered textures, 64x64 tiles with 24-bit z on chip	`docs/HARDWARE.md`
Peak per Raspberry Pi	1 Gpixel/s and 1.5 Gtexel/s	`docs/BENCH3D.md`
Scaler (HVS)	enlarges the framebuffer to the TV mode at no CPU cost	`docs/progress/graphics.md`
Output	mini-HDMI up to 1920x1080 at 60 Hz	`docs/HARDWARE.md`
Firmware vsync	not available on the Pi Zero; bm paces 60 fps with its timer	`docs/HARDWARE.md`

bm does not use OpenGL ES. It writes the V3D registers itself, with fixed-function drawing and its own QPU shaders (sources: `docs/PERFORMANCE.md`, section 7, for the direct-register fixed-function driver; `docs/DRIVERS.md` for the shaders).

[TO FILL: shots/pi-zero-w-bench-score.png]

ARM11 performance, as measured

All the numbers below were measured on a real Pi Zero W unless the text says otherwise. The frame budget at 60 fps is 16.7 ms and at 30 fps it is 33.3 ms (source: `docs/PERFORMANCE.md`).

Basic costs

Source: `docs/PERFORMANCE.md`, section 2, and `docs/HARDWARE.md`.

Operation	Cost
Fill 640x360, 32-bit	2.2 ms
Copy 450 KiB RAM to RAM, CPU / DMA	3.70 / 2.99 ms
Fill 450 KiB in RAM, CPU / DMA	1.02 / 0.85 ms
48-line strip RAM to screen, CPU / DMA	0.87 / 0.26 ms
Map + 256 sprites, direct / via a RAM buffer (benchmark `p`)	8.58 / 11.85 ms
Full frame copy through RAM, by DMA	14.06 ms (slower than the CPU)
256 sprites 16x16	about 0.9 ms
One simple Lua operation	about 100 ns (about 140,000 per frame)
Lua `fib(25)` / 1 million additions	83 ms / 104 ms

How many objects fit in a frame

The stress test counts the objects drawn per frame at 640x360, RGB565. Source: `docs/STRESS.md`, measure of 2026-10-05, ARM rows (the ARM rasterizer, bm3d 0.2).

Test	60 fps	30 fps	Cost per object
sprites 16x16 (C)	4454	9200	3.51 us
sprites 32x32 (C)	1501	3108	10.37 us
triangles 2D (about 170 px)	4726	9777	3.29 us
3D spheres of 96 triangles (C)	63 (2451 triangles)	201 (7910)	128.01 us
3D spheres, Gouraud	25 (990 triangles)	99 (3846)	212.54 us
3D spheres, textured	under 1	50 (1910)	259.37 us
quad 320x180, flat	11	25	22 ns per pixel
quad 320x180, flat without z	52	113	5 ns per pixel
quad 320x180, Gouraud	6	13	41 ns per pixel
quad 320x180, textured	3	6	81 ns per pixel

Test	60 fps	30 fps	Cost per object
sprites 16x16 (Lua)	1990	4110	7.85 us

What the numbers say (source: `docs/STRESS.md`):

- A sprite costs almost only its pixels: 16x16 is about 14 ns per pixel, and 32x32 has 4 times the pixels but costs 3 times as much. The limit is the bandwidth to video memory.
- Lua costs about 5 us more per sprite than C (8.5 against 3.5 in the September measure): the position maths and the `spr()` call.
- A 2D filled triangle of about 170 pixels costs 6.0 us in the September measure (`docs/PERFORMANCE.md`).

Cost per pixel of the ARM renderer

Source: `docs/PERFORMANCE.md`, section 5. A full-screen sphere is about 180,000 pixels.

Mode	Result on the Pi
Flat	fits in 16.7 ms: 60 fps with 15 spheres
Gouraud	over 16.7 ms: more than about 90 ns per pixel
Textured	over 33 ms: more than about 180 ns per pixel

Rule of thumb from the same file: Gouraud and textured triangles suit small and medium geometry, not full-screen fills. Draw big floors and skies flat and without z-buffer.

QEMU is not the Pi

QEMU runs about 2 times slower than the Pi on sprites (2133 against 4482 at 60 fps) and flat 3D (602 against 1195 triangles). It does not model the cache or the memory bus, so Gouraud and textures look much better there than on the real board (source: `docs/PERFORMANCE.md`, section 6). Always confirm a speed on the board.

Gaps in the ARM data

- **[TO FILL: the cost in cycles of the integer division (libgcc) on the ARM1176; the repo only says it is not in hardware]**

The GPU and the 2D

How bm draws in 2D

- All 2D is C code on the ARM; Lua only gives the logic (source: `docs/progress/graphics.md`).
- Cartridges draw in RGB565 (16 bit). The console and the menu use 32 bit (source: `docs/HARDWARE.md`). RGB565 halves the bytes per pixel; 32 bit doubles the frame copy time from about 6.5 ms to about 13 ms with the CPU (source: `docs/PERFORMANCE.md`).
- The framebuffer comes from the firmware mailbox. Its height is the screen height times the number of pages (1 to 3). A page flip changes the virtual offset (source: `docs/progress/graphics.md`).
- The cartridges draw directly into the video memory by default, because copying from RAM reads back the slow SDRAM: 8.58 ms direct against 11.85 ms through RAM (source: `docs/PERFORMANCE.md`). *Settings > Screen and sound > Game drawing (.bm)* switches between `Direct` and `Via RAM` (source: `src/kernel/settings.c`).
- bm uses triple buffering, which removed the tearing on the lower half of the menu (source: `docs/PERFORMANCE.md`).
- The 4x4 lighting grid, bilinear and dithered, is used by games with dynamic lights, at 320x180 (source: `docs/PERFORMANCE.md`).

Resolutions and the scaler

The cartridge resolutions are 640x360 (default), 480x270 and 320x180, plus the square 256x256 and 360x360 of the `.b16` edition. `screen(w, h)` switches at run time among 320x180, 384x216, 480x270, 640x360, 960x540, 1280x720 and 1920x1080 (sources: `docs/progress/graphics.md`, `docs/api/draw.md`).

The HVS scaler enlarges the picture to the TV. `scaling_kernel` in `config.txt` (at the root of the SD) picks its filter (source: `docs/progress/v3d.md`):

Value	Filter	Status in the repo
8	nearest neighbour	verified on the Pi, what bm uses today
6	`SINC_HAMMING_3PI`, the firmware default	verified on the Pi: looks good, soft, 640x360 to 1080p
1 to 5, 7	other sinc variants	listed by the firmware, not tried on the Pi

- [TO FILL: how the values 3, 5 and 7 of `scaling_kernel` look on the Pi; `docs/progress/v3d.md` lists them as untried]

What gfx16 offers

The 2D library is `src/bm/gfx16.h`. Every call is clipped to the clip rectangle and moved by the camera (source: `src/bm/gfx16.h`):

Group	Calls (C name; the Lua API is in `docs/api/draw.md`)
Shapes	`g16_cls`, `g16_pset`, `g16_pget`, `g16_line`, `g16_rect`, `g16_rectfill`, `g16_circ`, `g16_circfill`
Sprites	`g16_spr`, `g16_sspr`, `g16_sspr_zoom` (nearest pixel, bigger or smaller)
Map	`g16_map`, `g16_map_mask` (only cells whose tile has a flag)
Text	`g16_text`, `g16_text_scaled`
Light	`g16_light_*` (grid every 4 pixels), `g16_fade_*` (light by levels)
Sheets	`g16_sheet_alloc`, `g16_sheet_set`, `g16_sheet_update_cell`

Sprite sheets go up to 4096x4096 pixels. A sheet with at most 256 colours can be stored as SHEET8 (palette + runs), much smaller in the cartridge (sources: `docs/api/draw.md`, `docs/PERFORMANCE.md`).

The console's own 32-bit primitives are in `src/gfx/draw.h`: `gfx_clear`, `gfx_rect`, `gfx_sprite16` (16x16, one bit) and `gfx_text`. They draw on the framebuffer, and games do not use them.

DMA

- The DMA is used for the HDMI audio and for frame copies when enabled (source: `docs/HARDWARE.md`).
- For the 2D, DMA wins on strips and fills (48-line strip: 0.26 ms against 0.87 ms with the CPU) and loses on a whole frame (14.06 ms against 11.85 ms). bm copies whole frames with the CPU (source: `docs/PERFORMANCE.md`).
- The DMA settings are conservative on purpose: burst 8, high priority, `WAIT_RESP`. Aggressive settings locked the Pi up (source: `docs/PERFORMANCE.md`).
- At the end of a frame the 3D z-buffer is cleared by DMA while the next `_update` runs, about 1 ms saved at 640x360. `dma_zclear=0` turns it off (source: `docs/PERFORMANCE.md`).

2D drawn by the GPU

With `gpu3d_2d=1` (off when the key is missing) the GPU draws the 2D that comes after the 3D in the same job: `rect`, `pset`, horizontal and vertical lines, `spr`, `sspr` (also flipped and with whole-number zoom), the cells of `map()` and the text of `print`. The pixels are the same as the ARM's. Circles, oblique lines, triangles, `prompt` and a zoom that is not a whole number close the job, and the ARM draws them as before (sources: `docs/DRIVERS.md`, `src/kernel/settings.c`).

Measured on the Pi: the 2D on the GPU costs 9.5 ns per pixel, 19.6 ms at 1080p (source: `docs/progress/v3d.md`).

The GPU and the 3D

bm has two ways to draw 3D, behind the same functions. Your cartridge does not change.

The ARM renderer (r3d)

`src/bm/r3d.c` runs on the ARM. It transforms, lights, culls and clips with VFP floats and rasterizes in fixed point into a 16-bit z-buffer (1/z). It offers (source: `docs/progress/graphics.md`):

- flat and Gouraud shading (dithered 4x4), perspective-correct textures from the sprite sheet, near-plane clipping;
- sun and sky/ground light, specular, rim, up to 4 point lamps, fog;
- material bits per face: emissive, glossy, screen door, level of detail;
- planar shadows, a first-person layer, points, lines and billboards;
- rigid skinning (one bone per vertex).

It is the driver bm3d 0.2. It draws everything when `gpu3d=0`, in QEMU, when the GPU does not answer or fails the boot probe, and for what the GPU cannot do (source: `docs/progress/graphics.md`).

The V3D path (gpu3d)

`src/gpu/gpu3d.c` is a backend of r3d. The ARM still transforms, lights and clips; the V3D fills the pixels (source: `src/gpu/gpu3d.h`, `docs/GPU-BENCHMARK-REPORT.md`).

- The V3D draws in tiles of 64x64 pixels: a binning list sorts the triangles per tile, then every tile is drawn on the chip (colour, 24-bit depth) and stored (source: `src/gpu/v3d.h`).
- r3d collects the triangles into one GPU job per frame where possible. Tiles are loaded from the page, so the 2D drawn before the 3D stays under it.
- The shaders are small QPU programs written in `tools/qpuasm.py` and generated into `src/gpu/shaders.h`.
- Differences from the ARM renderer: smooth Gouraud without dither noise, 24-bit depth instead of 16-bit 1/z, no `zclear()` needed for the depth buffer (it starts from zero each frame) (sources: `docs/GPU-BENCHMARK-REPORT.md`, `docs/api/light3d.md`).
- A vertex shader (on by default) can place the vertices of whole meshes on the GPU, so the ARM sends only matrices and lights.
- The pixels match the ARM renderer to within 0.1% in the emulator tests (source: `docs/GPU-BENCHMARK-REPORT.md`).

When each is used

Situation	Who draws
A V3D answers and `gpu3d` is not 0	the GPU (default)
`gpu3d=0` in `bm/config.txt`, or *Settings* > *Screen and sound* > *3D of the games: ARM*	the ARM
QEMU	the ARM
The GPU times out or a boot probe fails	the ARM takes over by itself, with a line in the log
2D that the GPU cannot reproduce exactly (list above)	the ARM, after the job

Situation	Who draws
A triangle that crosses the vertex guard band	clipped by the ARM

Sources: `docs/progress/graphics.md`, `docs/GPU-BENCHMARK-REPORT.md`, `docs/DRIVERS.md`.

Optional features are probed at boot. The probe checks that the GPU draws what the ARM would, and switches the feature off by itself if not. The status line (*Dev* > *GPU test*, or `g` in the monitor) says what was kept, for example `vertex shader yes` or `ETC1 textures yes (utiles 2x4, bytes BE)` (source: `docs/DRIVERS.md`).

Options

Each option is a key in `bm/config.txt` and a row in *Settings* > *Screen and sound*. The default is what applies when the key is missing (source: `src/kernel/settings.c`, `docs/DRIVERS.md`).

Key	What it does	Default
`gpu3d`	the GPU draws the 3D (0 = ARM)	on where a V3D answers
`gpu3d_aa`	4x MSAA	off
`gpu3d_vs`	vertex shader: 0 ARM, 1 scenery, 2 all models	2
`gpu3d_queue`	1: the job starts and the next `_update` runs meanwhile; 2: two jobs in flight	off
`gpu3d_later`	each frame shown a frame later (needs `gpu3d_queue=2`)	off
`gpu3d_2d`	2D over the 3D inside the GPU job	off
`gpu3d_filter`	bilinear textures (the look changes)	off
`gpu3d_wc`	job memory uncached	off
`gpu3d_tex16`	16-bit textures (RGB565; cut-out sheets as RGBA5551)	off
`gpu3d_etc1`	opaque sheets as ETC1 (sets `gpu3d_tex16` too)	off
`gpu3d_fs2`	textured pixel shaders with two threads	on (`0` turns it off)
`gpu3d_sort`	meshes nearest first, triangles in vertex-cache order	off
`gpu3d_attr`	compact vertex attributes	off
`r3d_fast`	ARM renderer: one matrix and light in model space (pixels no longer bit-identical)	`r3d_fast=1` to turn on
`r3d_impostor`	far models as one quad (2 to 64 pixels of a 1080-line screen)	off

In the Settings the rows are named *3D of the games*, *3D anti-aliasing*, *3D vertices*, *3D frame queue*, *3D frame shown*, *2D over the 3D*, *3D textures*, *3D job memory*, *3D pixel shaders*, *3D draw order*, *3D corners*, *3D impostors* and *3D on the ARM* (source: `src/kernel/settings.c`).

Limits you meet:

- MSAA works only in jobs without a kept z-buffer. A cartridge that draws 3D, then 2D, then 3D again in one frame stays without anti-aliasing (source: `docs/api/light3d.md`).
- Textures with sides that are multiples of 32 go to the GPU in T-format, which is faster to read (source: `docs/api/light3d.md`).
- ETC1 needs sheet sides that are multiples of 64x128 (or 128x64) texels (source: `docs/DRIVERS.md`).
- On the Pi, 16-bit and ETC1 sheets gave the same speed as 32 bit in the 3D Bench, within 1 to 6%, because small sheets fit in the texture cache (source: `docs/progress/v3d.md`).
- Draw all the 3D first and then the HUD. 3D drawn after 2D in the same frame forces the GPU to keep the z-buffer, about 1 MB written and read again per frame (source: `docs/api/light3d.md`).

What the GPU gives, as measured

Source: `docs/GPU-BENCHMARK-REPORT.md`, section 4, at ARM 1000 MHz, core 250, V3D 250, SDRAM 400.

Measure	Result
Small triangles	3.0 million per second (20,000 prepared in 7.7 ms)
Fill rate	811 Mpixel/s (software fill about 45 Mpixel/s)

Stress test, most objects per frame at 640x360:

Scene	ARM, 60 fps	GPU, 60 fps	GPU, 30 fps
Flat spheres	69 (2679 triangles)	182 (7142)	389 (15346)
Gouraud spheres	18 (703)	170 (6673)	364 (14360)
Textured spheres	under 1	156 (6094)	333 (13135)
Full-screen flat quads	11 (22 ns/px)	200 (1 ns/px)	430
Full-screen Gouraud quads	5 (49 ns/px)	199 (1 ns/px)	427

Real programs, same file:

Program	ARM	GPU
Texture Room HD 640x360	25.3 ms (8 crates)	5.8 ms (32 crates, 573 triangles)
Chaos Kitchen	14.1 ms	6.8 ms (758 triangles)

The ARM instructions per frame fall too, in a QEMU host simulation (not measured on the Pi): Texture Room, 8 crates, 320x180, 446 triangles, goes from 3.86 million to 0.62 million (source: `docs/GPU-BENCHMARK-REPORT.md`, section "Host Simulation Metrics (qemu-arm)").

With the GPU the spheres are limited by the ARM (about 2 us per triangle for transforms and vertices), while the quads are limited by the V3D (source: `docs/STRESS.md`). The measure of 2026-10-05 (bm3d 4.2,

same file) gave 207 GPU spheres at 60 fps (8131 triangles), 159 Gouraud, 165 textured, and a textured 256x256 quad at 3 ns per pixel (91 quads), with 4x MSAA costing almost nothing (204 spheres).

A real game: Overbit

Overbit is bm's most demanding 3D title. Source: `docs/GPU-BENCHMARK-REPORT.md`, section 6 (estimates for a 10-bot skirmish on the Pi Zero W):

Preset	ARM	GPU
LOW	36.0 fps	49.6 fps
MEDIUM	33.7 fps	45.0 fps
HIGH	29.8 fps	35.5 fps
ULTRA	25.1 fps	33.8 fps
EXTREME	24.7 fps	31.0 fps

The GPU cuts the ARM instructions per frame by 23% to 32% across these presets. At 1080p a played match is limited by fill: the GPU takes 20.4 ms for 6105 triangles and 10.1 ms at 720p. Fewer pixels are the lever, not fewer triangles (source: `docs/progress/v3d.md`).

- [TO FILL: Overbit played on the Pi with `gpu3d_attr=1`, `gpu3d_tex16=1` and `gpu3d_etc1=1` against all off; `docs/progress/v3d.md` lists it as still to measure]

The bm3d drivers

One name, two drivers

3D goes through **r3d** (scene, transforms, light, the ARM rasterizer) and **gpu3d** (the V3D backend with its shaders). They share one version, **bm3d X.Y**. The current one is bm3d 7.3 (source: `src/gpu/version3d.h`). You see it in *Settings* > *System* > *3D driver*, in the GPU test, in the log when a game starts, in the 3D Bench and as the fourth value of `gpu3d()` (source: `docs/DRIVERS.md`).

The versions that matter

Source: `docs/DRIVERS.md`.

Version	What it brought
0.2	ARM only: fixed-point edges, specialised texture loops, z cleared by DMA (69 spheres at 60 fps)
1.0	the GPU first draws triangles (NV shaders)
2.0	clean pages, T-format textures and MSAA (182 spheres at 60 fps)
2.1	Overbit on the GPU: screen-door faces, precomputed RGB light and fog, shadows, 3D effects
3.0 to 3.4	the vertex shader places the scenery, then lit models, bones and skins
4.0, 4.1	the queued frame: the job starts and the next `_update` runs while the GPU draws
4.2	screens up to 1920x1080; `cls()` done by the GPU job
5.0	meshes up to 65535 vertices and 65535 faces, indexed on the GPU
5.1	two jobs in flight
5.2, 7.2	16-bit textures (opaque sheets, then cut-outs)
5.3, 6.4	textured shaders with two threads, on by default from 6.4
5.4, 5.6	draw order, nearest first and vertex-cache order
5.5	`visible3d()` and `pvs3d()`
6.6	the vertex shader on by default for all models
6.9	each frame shown a frame later (option)
7.0	textured faces drawn as flat colours (`draw3d` flag 128, `textures3d(false)`)
7.1	compact attributes
7.3	opaque sheets as ETC1

`docs/DRIVERS.md` also explains how the settings reproduce older versions on today's code, so you can compare them on the same Pi: `gpu3d=0` is 0.2, `gpu3d_vs=0` is 2.1, `gpu3d_vs=1` is 3.0, `gpu3d_vs=2` is 3.4, and so on. Versions 0.1 and 1.0 no longer run; the repo keeps the numbers measured with them.

The 2D side

For a game, the 2D driver is `gfx16`, with the calls of `docs/api/draw.md`: `cls`, `screen`, `pset`, `pget`, `line`, `rect`, `rectfill`, `circ`, `circfill`, `tri`, `print`, `font`, `prompt`, `camera`, `clip`, `spr`, `sspr`, `zspr`, `map`. `cls()` with the 3D on the GPU is free: the GPU job clears the page to that colour. If 2D comes before the 3D, the ARM fills it as always (source: `docs/api/draw.md`).

The 3D side

The functions are described in `docs/api/light3d.md`. The ones that matter for performance:

Function	Use
`mesh(v, f, [uv])`, `model(name)`	meshes, up to 65535 vertices and 65535 faces
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	flags: 1 no z-buffer (floors, skies: faster), 2 no light, 4 smooth, 8 shadow, 64 in front, 128 flat textures; they add up
`camera3d`, `light3d`, `sky3d`, `fog3d`, `lamp3d`	camera and light
`zclear()`	clears the z-buffer
`visible3d(x, y, z, r)`, `pvs3d(t)`	skip what cannot be seen before drawing it
`textures3d(false)`	every `draw3d` draws textured faces as flat colours
`stat(9)`	1 when the GPU does the 3D
`stat(16)`	the GPU job time of the last frame in ms; read with the queue on, it includes the wait, so trust it only without the queue

How a game chooses

Without doing anything, the game gets the console's settings. A game can ask for its own with `gpu3d([on, aa, vs, queue, impostor, later])`. It returns `on, aa, vs, version, queue, impostor, later`, and it changes the choice only for this cartridge (source: `docs/api/light3d.md`):

- `on` stays `false` if there is no GPU (QEMU, `gpu3d=0`) and in 256x256 games (the GPU writes whole pages);
- `aa` stays off if MSAA cannot be used, `vs` stays off if the boot test did not see the vertex shader work;
- `queue` `true` asks for the console's level, `false` for none; `later` works only with `queue` 2 and never in a game played online;
- `version` is the bm3d version this choice reproduces (`"0.2"` for the ARM, `"2.1"`, `"3.0"`, `"3.4"`, `"4.1"`, `"6.9"`).

COPY AND PASTE

```
-- ask for the GPU with the vertex shader for every model
gpu3d(true, false, 2)
-- impostors for far models only, leaving the rest as it is
gpu3d(nil, nil, nil, nil, 16)
-- read what is in force
local on, aa, vs, version = gpu3d()
```

Measure on your own board

Do these on a Pi Zero W. You can run everything from the menu and read the reports on the SD card; no serial cable is needed. To put a new kernel on the SD you copy `dist/kernel.img` there, with `./easy_install.sh` (source: `docs/BENCH3D.md`, `docs/STRESS.md`).

The 3D Bench

It is bm's single benchmark of the 3D. Each test grows its load until a frame passes 40 ms. It runs every driver profile (ARM, GPU, GPU+AA, GPU+VS, GPU+VS+FS2 and others) and interpolates the loads at 60 fps and 30 fps. Source: `docs/BENCH3D.md`.

Try it, step by step

1. Put the SD card in the Pi Zero W and switch it on. Wait for the bm menu.
2. Open the Dev tab and choose **3D Bench**. You can also press `j` in the serial monitor.
3. Leave it alone. The Pi report of 2026-10-05 ends with `total 79 s`. Each step measures 6 frames and keeps the middle one, so a single slow frame does not move the result.
4. Use left and right to change page. The first page is the score. Press B (or Esc) to leave.
5. Take the SD card out and open `bm/bench/` on it: the run saved `3D0001.TXT`, `3D0002.TXT`, and so on.
6. The next run reads the last report and compares against it. To keep a history, copy the files to `docs/bench/` in the repository.

TRY IT

Run the 3D Bench twice. On the second run the pages show white ticks at the values of the first one, and a column "x last".

[TO FILL: shots/pi-zero-w-bench-test.png]

What you read (source: `docs/BENCH3D.md`):

- The **score**: 1000 is bm3d 2.1 on the Pi Zero W, 2000 is a driver twice as fast in every test.
- The **triangles per frame at 60 fps** of the `mix` scene, with the games' driver.
- For each test and profile: the load at 60 fps, the ms (average and worst), the triangles, the GPU ms, and the ARM instructions and data cache misses per frame (from the ARM1176 counters, only on the real Pi).
- The `machine` line with the clocks, the temperature and the throttling flags.

A part of the bench, from the monitor

The monitor's line runs only some tests and profiles (source: `docs/BENCH3D.md`).

COPY AND PASTE

```
:gpu; b3d tests=match,quad_tex profiles=GPU,FS2; send
```

From the PC, with the board on the network:

COPY AND PASTE

```
./easy_install.sh line "gpu; b3d; send"
```

The console writes what it does until it says `the line is done`.

The cost of a pixel

Source: `docs/progress/v3d.md` and `docs/BENCH3D.md`. The `P` lines of the report give the ms to fill 640x360, 1280x720 and 1920x1080.

COPY AND PASTE

```
./easy_install.sh line "gpu; b3d tests=quad_flat,quad_tex,bilinear,gpu2d profiles=ARM,GPU,FS2,2D,VSF; se
```

The GPU test and the stress test

1. Open the Dev tab and choose **GPU test** (or `g` in the monitor). It runs the driver's steps, prints the status line and the probe log, and reports the triangle rate and the fill rate (sources: `docs/DRIVERS.md`, `docs/GPU-BENCHMARK-REPORT.md`).
2. For the stress test of sprites and 3D, build the dedicated kernel on the PC and copy it to the SD (source: `docs/STRESS.md`):

COPY AND PASTE

```
cd ~/bm && git pull
make sdcard-stress
cp dist/kernel.img /path/to/the/SD/ && sync
```

3. Start the Pi. After the system summary the tests run, about 30 to 60 s in all, and the table of results stays on the screen: a photo is enough.
4. To go back to the normal kernel, run `make sdcard` and copy `dist/kernel.img` again.

If you have a serial cable, the stress test starts from the monitor with the `S` key.

Compare the drivers on the same board

The settings reproduce the older drivers (see chapter 5), so you can run the same bench with each. Use `set key=value` in the monitor to change a key of `bm/config.txt` until the next reboot and `save` to keep it (source: `docs/BENCH3D.md`). Every report prints the 3D options in force on one `options` line, for example `bm3d 7.3 gpu3d=on vs=2 aa=0 queue=2 wc=0 filter=0 tex16=0 fs2=1 sort=0 attr=0 etc1=0`.

BEWARE

Do not compare numbers from kernels with different `options` lines, or with different clocks on the `machine` line. The repo saw `quad_flat` run at 175, 91 and 70 in different runs with the same driver, and tied it to the core and V3D clocks (source: `docs/BENCH3D.md`).

No report for a stopped test

A test that you stop (Start+Select, Ctrl+Esc or PS after the frame on screen) leaves no report (source: `docs/progress/graphics.md`).

If the current numbers are missing

- [TO FILL: a complete 3D Bench report of the Pi Zero W at bm3d 7.3; the only full report in `docs/bench/` is at bm3d 4.2 (kernel v0.2.0)]

Limits and tips

Limits

- One core does everything: game, graphics, audio, input. The budget per frame is 16.7 ms at 60 fps (source: `docs/PERFORMANCE.md`).
- Memory bandwidth is the first bottleneck, not RAM size. Every extra byte per pixel costs (source: `docs/PERFORMANCE.md`).
- The 3D on the GPU is limited by the ARM for small triangles (about 2 us per triangle) and by the V3D for big ones (source: `docs/STRESS.md`).
- Divisions: use them per triangle at most, never per scanline or pixel. A denormal float traps unless RunFast is on; bm turns RunFast on at boot (source: `docs/PERFORMANCE.md`).
- The firmware vsync is not available on the Pi Zero (source: `docs/HARDWARE.md`).
- One USB OTG port with one HID device plus a mouse; up to 4 DualShock 4 over Bluetooth (source: `docs/HARDWARE.md`).
- The 3D is not OpenGL: it is bm's own fixed-function driver. Running your own programs on the QPUs (particles, full-screen effects) is listed as not started (source: `docs/progress/v3d.md`).
- No interrupt for the V3D: the driver polls (source: `docs/progress/v3d.md`).

Tips

- Pick the resolution first. 640x360 is 11% of the pixels of 1080p; 480x270 is an exact 4 times integer scale to 1080p (sources: `docs/HARDWARE.md`, `docs/GPU-BENCHMARK-REPORT.md`). Above 640x360 on the Pi the ARM rasteriser pays for every pixel and the GPU is needed (source: `docs/api/draw.md`).
- Draw floors and skies with `draw3d` flag 1 (no z-buffer).
- Draw all 3D first, then the HUD.
- Start the frame with `cls()`: with the GPU the page is not read again.
- Use `visible3d()` to skip hidden characters.
- Use `r3d_impostor` for crowds of far models.
- Use sheets with sides that are multiples of 32 so the GPU reads them in T-format.
- Test on the board. QEMU does not model the cache or the memory bus (source: `docs/PERFORMANCE.md`).
- When numbers jump between runs, read the `machine` line: the repo tied slow quads to the core and V3D clocks (source: `docs/BENCH3D.md`).

The RGB30, where the repo compares

The PowKiddy RGB30 (RK3566, AArch64) runs the same 3D Bench. Its report in the repo is `docs/bench/20261005_bench3d_rgb30_v0.2.1.txt`, bm3d 4.2, and has only the ARM profile: "no GPU driver on the RGB30 yet". The Pi report is `docs/bench/20261005-082943_bench3d_pi-zero-w_v0.2.0.txt`, the same bm3d. The ARM profile (bm3d 0.2) loads at 60 fps:

Test	Pi Zero W, ARM	RGB30, ARM
spheres	72.3	180.1
spheres_smooth	21.9	100.3
heroes	3.9	8.6
draws	453.1	899.9
quad_flat	11.7	16.2
quad_tex	3.4	8.0

A value of -1 means the first step already did not fit in 16.7 ms: `spheres_tex` on the Pi is -1 and 45.4 on the RGB30; `match` is -1 on the Pi and 1.3 on the RGB30.

Since then the repo has started a driver for the RGB30's Mali-G52 GPU. It powers the GPU, runs jobs and clears a surface; a triangle is written and not yet tried on the device. The 3D of the RGB30 is still `r3d` on the ARM (source: `docs/progress/graphics-mali.md`). So on the RGB30 there are no GPU numbers to compare with the Pi's V3D.

Open points

- **The board**: the effect of `force_turbo=1` or of the 400 MHz core clock on the 3D Bench; `docs/STRESS.md` says "to be tried"
- **The board**: picture to take: shots/pi-zero-w-bench-score.png
- **ARM11 performance, as measured**: the cost in cycles of the integer division (libgcc) on the ARM1176; the repo only says it is not in hardware
- **The GPU and the 2D**: how the values 3, 5 and 7 of `scaling_kernel` look on the Pi; `docs/progress/v3d.md` lists them as untried
- **The GPU and the 3D**: Overbit played on the Pi with `gpu3d_attr=1`, `gpu3d_tex16=1` and `gpu3d_etc1=1` against all off; `docs/progress/v3d.md` lists it as still to measure
- **Measure on your own board**: picture to take: shots/pi-zero-w-bench-test.png
- **Measure on your own board**: a complete 3D Bench report of the Pi Zero W at bm3d 7.3; the only full report in `docs/bench/` is at bm3d 4.2 (kernel v0.2.0)